

eduRAG: A Confidence-Weighted GraphRAG Framework for Multi-Hop Educational Question Answering

Dr. Sarika Zaware
Dept. of Computer Engineering
AISSMS IOIT, Pune, India
sarika.zaware@aissmsioit.org

Aditya Kokate
Dept. of Computer Engineering
AISSMS IOIT, Pune, India
adityakokate1.3.5@gmail.com

Ayush Bhujbal
Dept. of Computer Engineering
AISSMS IOIT, Pune, India
ayushbhujbal17@gmail.com

Yash Divekar
Dept. of Computer Engineering
AISSMS IOIT, Pune, India
yashdivekar19@gmail.com

Sameer Kulkarni
Dept. of Computer Engineering
AISSMS IOIT, Pune, India
sameerkulkarni021@gmail.com

Abstract—Most retrieval-augmented generation (RAG) systems built for education share the same three weak spots: the triples they extract from messy documents are noisy, multi-hop reasoning across linked concepts breaks down, and dense retrieval gives learners no real way to see why an answer was chosen. We built *eduRAG* to attack these problems together. The idea is simple: treat the confidence of every extracted triple as a first-class signal, and let it flow through the whole pipeline rather than dropping it after extraction. Four pieces make this work. First, a multi-axis scorer rates each triple on factuality, specificity, and coherence. Second, a Confidence-Weighted Leiden (CW-Leiden) variant uses those scores as edge weights when detecting communities. Third, a language model drives traversal step by step through a sandboxed REPL, so depth adapts to query complexity instead of being hard-coded. Fourth, several seed entities are explored in parallel, and their paths are merged using a confidence-aware overlap rule. Across three real document artifacts, CW-Leiden gives a mean weighted modularity of 0.9248 and mean community coherence of 0.3173. On our custom Educational QA corpus the system reaches ROUGE-L 0.442 (a 25.6% jump over a naive RAG baseline), BERTScore F1 of 0.912, and METEOR of 0.385. *eduRAG* runs as a full-stack platform—FastAPI on the backend, React on the frontend, ChromaDB for vectors, and pluggable LLM backends (Ollama, Groq, OpenAI, Anthropic, Gemini). Every variant we report can be reproduced from a single YAML file.

Index Terms—Retrieval-Augmented Generation, GraphRAG, Educational Technology, Knowledge Graphs, Multi-hop Reasoning, Confidence Scoring, Community Detection, Graph-Grounded QA

I. INTRODUCTION

A. Why this problem matters

Classrooms are not what they used to be. We have moved past the days of slide decks. These days students want a system where they can ask the classrooms tough questions that have many layers and get explanations that actually make sense. The classrooms should give explanations that are based on the lecture notes, the textbooks and the papers that the students uploaded to the

classrooms that morning. The standard RAG pipelines are not good enough for the classrooms in this case. The RAG pipelines have a problem because they focus on finding similarities in parts of the text and this does not work well for the classrooms, for four main reasons related to the classrooms. Standard RAG pipelines, which lean on chunk-level similarity, struggle here for four related reasons.

Extraction has a lot of noise. When a chunk has a definition, a worked example and a paragraph of text triple extractors make up. Invent relations that are not fully supported. Multi-hop reasoning does not work well. A fixed depth, for traversal. Brings in too much irrelevant information or does not reach the required chain of information that the question relies on. It is hard to understand how the system works. A long list of retrieved passages does not help a learner understand why those passages were chosen or which concept link was important. The educational field is very demanding. Educational queries use a lot of terms and require prior knowledge that generic retrievers do not have. The field of education requires an understanding of specific terminology and concepts. Retrievers often miss information because they lack the necessary background knowledge.

Graph-based retrieval helps with all four. By organising what was extracted as a typed entity-relation graph, it lets the system reason explicitly and lets the learner trace the path. But existing GraphRAG systems—including the well-known work of Edge et al. [6]—weight every triple the same when they cluster or traverse. That seems wrong to us. A clean, well-supported triple from a textbook should pull harder on the structure than something an LLM half-guessed from a discursive paragraph. *eduRAG* is built on that intuition, and pushes it through every stage of the pipeline.

B. What we contribute

Four specific things separate eduRAG from earlier GraphRAG work:

- **C1 – Confidence-scored extraction.** A composite score weighting factuality (0.40), specificity (0.35), and coherence (0.25) drops the weakest 20–40% of candidate triples before they ever touch the graph. The retained set has a mean confidence of 0.6479.
- **C2 – Confidence-weighted community detection.** CW-Leiden treats edge confidence as modularity weight, reaching a weighted modularity of 0.9248 and mean community coherence of 0.3173.
- **C3 – LLM-guided traversal.** Rather than hard-coding the hop count, an LLM issues graph operations through a sandboxed REPL (15 steps max, 30 s per step). Depth becomes a function of the question.
- **C4 – Parallel multi-seed traversal.** Up to five seeds are explored at once and their paths merged through a confidence-weighted overlap rule. Coverage goes up; latency does not.

C. Scope and reproducibility

eduRAG is a working platform, not just a paper artifact. The backend is a FastAPI service exposing upload, chunking, querying, and analytics endpoints. The frontend is a React app for student interaction and graph exploration. The Python core swaps between Ollama, Groq, OpenAI, Anthropic, and Gemini backends, and stores vectors in ChromaDB. Every one of the six variants below is defined by a released YAML file—change one line, rerun, get the same numbers.

II. RELATED WORK

A. RAG, briefly

Lewis et al. [1] put RAG on the map by pairing a retriever with a generator and showing real gains on open-domain QA. Later work added dense retrievers [2], neural rerankers [3], and self-critique loops [4]. None of it changes the basic move: pull unstructured chunks, hand them to the model, hope the chain holds. For multi-step questions, hope is not a strategy.

B. Graphs as a way out

Knowledge graphs have a long history in symbolic reasoning, and graph neural networks [5] brought that tradition into the learned-representation era. Microsoft Research’s GraphRAG [6] is the closest neighbour to our work—it builds an entity-relation graph from LLM extractions and clusters it into communities for hierarchical retrieval. Tan et al. [7] added adaptive traversal on top. Both, though, treat every triple as equally trustworthy and rely on heuristic or fixed-depth exploration. Those are the two assumptions we wanted to break.

C. Confidence as a signal, not a filter

Confidence in relation extraction is not new. People have used model logits, multi-pass scoring [8], and ensembles [9], and the downstream cost of low-confidence

outputs is well documented [10]. What is new in eduRAG is where the signal lives. Instead of using confidence only to throw away bad triples, we let it shape how communities form and how traversal expands.

D. Community detection

Leiden [11] fixed Louvain’s worst pathology—disconnected communities—and is now the default for most graph clustering work. Weighted modularity is standard in network science [12], and edge-weight-aware clustering reliably beats unweighted versions in biological and social networks. Plugging extraction confidence into Leiden for a RAG knowledge graph is, as far as we can tell, new.

III. PROBLEM FORMULATION

Let $D = \{d_1, d_2, \dots, d_n\}$ be a corpus of educational documents. Each d_i is sliced into overlapping 200-token chunks (64-token overlap), and from every chunk c_{ij} an extractor produces up to $K = 16$ triples $T_{ij} = \{(s, r, o)\}$. The candidate set is $T_{cand} = \bigcup_{ij} T_{ij}$. We then build a directed graph $G = (V, E)$ in which nodes are entity mentions and edges are triples carrying both a relation label and a confidence weight.

Given a query q , the system has to produce an answer $\hat{a}$ that is grounded in graph-retrieved triples, holds together across multiple hops, exposes the reasoning path, and never silently leans on a low-confidence extraction.

Four research questions structure the evaluation:

- **RQ1.** Does C1 actually clean up the graph and improve answer quality?
- **RQ2.** Does C2 produce more coherent communities than vanilla Leiden?
- **RQ3.** Does C3 beat fixed-hop traversal on multi-hop educational questions?
- **RQ4.** Does C4 actually purchase insurance coverage or does it simply use up more computing resources?

IV. SYSTEM ARCHITECTURE

A. Pipeline

The pipeline has nine stages. You can configure each one using YAML. So documents come into the pipeline. The pipeline breaks them down into smaller pieces. Then the pipeline extracts information from these pieces gives them a score and uses this information to build a graph. The pipeline uses CW-Leiden on this graph. When you ask the pipeline a question ChromaDB gives you the 5 seed entities that are most relevant. Then the pipeline collects triples that are related to these entities. It only looks at a certain amount of information which is 3,000 tokens. The pipeline uses this information to generate an answer to your question, about the documents and the graph and the entities.

B. Chunking and extraction

The chunker uses a sliding window of 200 tokens. There is a 64-token overlap. This overlap helps keep concepts that’re on both sides of a boundary from being split awkwardly.

We support methods for extraction. These methods are:

- * An LLM-based prompt
- * REBEL-large
- * spaCy `en_core_web_lg`

You can select any of these methods from the configuration. We drop entities that’re shorter, than four characters. This helps avoid pulling in punctuation and short tokens.

C. Confidence scoring (C1)

Every triple gets a score that is made up of parts and this score is calculated in groups of 8 at a time. This is done to make sure that things happen quickly and do not take long. Every triple gets this score.

$$\text{Conf}(t) = 0.40 \cdot F(t) + 0.35 \cdot S(t) + 0.25 \cdot C(t) \quad (1)$$

The function $F(t)$ checks if we can really see the triple in the piece of information. The function $S(t)$ checks if the information is specific enough to be of any use. For example if the triple is about things and stuff it is not specific enough. The function $C(t)$ checks if the subject, the relation and the object make sense together.

If the result is than 0.15 we throw it away. This usually removes around 20 to 40 percent of the possibilities we are considering. It makes the average confidence of the remaining ones higher it goes up, to 0.6479.

D. Graph construction and CW-Leiden (C2)

When we talk about surviving triples they land in a NetworkX directed graph. This is where fuzzy matching comes in which basically means it collapses duplicate entity mentions in the NetworkX directed graph.

The embeddings we use come from sentence-transformers/all-MiniLM-L6-v2, which’s 384-dim and uses CUDA acceleration. We store these embeddings in ChromaDB.

For community detection we use CW-Leiden with a resolution of 1.0. We run it for ten iterations with a seed of 42, for the NetworkX directed graph.

$$Q_w = \frac{1}{2m_w} \sum_{ij} \left(w_{ij} - \frac{k_{iw}k_{jw}}{2m_w} \right) \delta(c_i, c_j) \quad (2)$$

$w_{ij} = \text{Conf}(t_{ij})$ is the edge weight, k_{iw} is the weighted degree, m_w the total weight. Community coherence is the average pairwise cosine similarity of entity embeddings inside each detected community—a basic but honest measure of whether the cluster actually represents one idea.

E. LLM-guided traversal (C3)

This is the part that surprised us most. Instead of a hard-coded $k = 3$ expansion, we let an LLM run graph operations through a sandboxed REPL: `get_neighbors`, `get_path`, `get_community`, `get_subgraph`, `get_community_members`. After each call it sees the triples collected so far and the nodes already visited, then decides what to do next. A confidence threshold of 0.3 is enforced inside traversal, each step times out at 30 seconds, and the whole run terminates on a DONE signal or after 15 steps. Those bounds matter for interactive use—they cap worst-case latency without forcing the model to give up too early.

F. Parallel multi-seed traversal (C4)

For questions that name several entities at once, traversal launches from up to five seeds concurrently with a 60-second async timeout. A node “converges” when at least two independent traversals reach it, and convergence is weighted by the confidence of the paths that got there. The side effect is a natural ranking: nodes hit by several high-confidence paths float to the top during context assembly, without any extra scoring step.

G. Context assembly and generation

Collected triples are deduplicated at a token-overlap threshold of 0.92, sorted by confidence, and trimmed to 3,000 tokens. The generation prompt is strict: answer only from the graph context, keep an academic register, and say so explicitly when something is missing.

V. EXPERIMENTAL METHODOLOGY

A. Datasets

We use three datasets that cover types of questions and documents.

These datasets are important for the Educational QA and other question answering tasks.

The three datasets are:

- **QA (Custom)**. This dataset is made up of lecture notes, textbooks and technical papers. It has a lot of Educational QA documents. 45 Percent are papers 32 percent are lecture notes and 23 percent are textbooks. The Educational QA documents are, about machine learning, deep learning and natural language processing. The Educational QA dataset has questions that need one to three hops to answer.
- **MuSiQue** [16]. This dataset has 500 questions to test how well our system works. The MuSiQue dataset has questions that need two to four hops to answer. The answers are found on Wikipedia.
- **2WikiMultiHopQA** [17]. This dataset also has 500 questions to test our system. The 2WikiMultiHopQA dataset has questions that need two to three hops to answer. The answers are found in pairs of Wikipedia articles.

B. Variants

The variants of the model are tested one by one. There are six variants of the model and they all have some things in common. They use seed 42 `ollama/llama3.2:3b` and temperature 0.0. The only difference between the variants of the model is which contributions are turned on. The contributions are called C1, C2, C3 and C4. You can see the details, in Table I.

TABLE I
ABLATION VARIANTS

Variant	C1	C2	C3	C4
Naive RAG	–	–	–	–
Standard GraphRAG	–	Leiden	Fixed-3	–
+C1	✓	Leiden	Fixed-3	–
+C1+C2	✓	CW-Leiden	Fixed-3	$k \leq 4$
+C1+C2+C3	✓	CW-Leiden	RLM	–
Full eduRAG	✓	CW-Leiden	RLM	$k \leq 5$

C. Metrics

We measure the quality of answers using ROUGE-L, BERTScore F1 and METEOR. To evaluate the quality of graphs we use modularity, which is denoted as Q_w , as well as mean community coherence and mean triple confidence. The metrics also include per-query latency, the number of tokens used the number of nodes visited and the maximum traversal depth of the graph.

D. Implementation

When we run the system locally we use Ollama llama3.2:3b with the temperature set to 0.0 the number of tokens set to 1024 and a timeout of 120 seconds. For summarisation benchmarks we use Gemini 1.5 Pro with text-embedding-004 and SAEOCR version 1.2. We create graphs using NetworkX and use `leidenalg` for community detection, in these graphs.

VI. RESULTS AND ANALYSIS

A. Graph quality

Table II shows some numbers about the graphs we looked at in three things. The HTML/CSS guide makes the graph by a lot. The two graphs from images are not as big. We are more sure about what is in them. Graph quality is pretty good because the mean weighted modularity is 0.9248, for all three graphs. This means CW-Leiden does a job of separating groups in the graph quality no matter how big the graph is.

TABLE II
VERIFIED GRAPH QUALITY METRICS

Artifact	Nodes	Edges	W. Mod.	Coher.
HTML/CSS Guide	329	235	0.9715	0.2892
Image Doc (1)	30	16	0.9171	0.3388
Image Doc (2)	27	16	0.8858	0.3239
Mean	–	–	0.9248	0.3173

B. Answer quality

Table III shows the results for the Educational QA corpus we put together. If we look at what happens when we take away each part it is clear what each part does. Each part makes a difference that we can see. When we use all the parts together the result is better, than using any one part by itself. The Educational QA corpus is what we are talking about here. We can see that using all the parts together makes the Educational QA corpus work better.

TABLE III
ANSWER QUALITY ON THE EDUCATIONAL QA CORPUS

Metric	eduRAG	Baseline	Gain	95% CI
ROUGE-L (F1)	0.442	0.352	+25.6%	± 0.012
BERTScore (F1)	0.912	0.842	+8.3%	± 0.012
METEOR	0.385	0.338	+13.9%	± 0.012

C. Vision and OCR

Our SAEOCR version 1.2 can read notes from students with an accuracy of 91.8 percent. In comparison Tesseract version 5 has an accuracy of 57.5 percent. This 34 percent difference in accuracy makes eduRAG really useful for the types of materials that students usually upload. These materials include scanned textbook pages with notes in the margins photos of problem sheets and diagrams with annotations.

D. Latency and scale

C4s way of executing tasks in parallel helps us cover ground at a lower cost. This is because tasks are done at the time instead of one after the other. C3 costs more than the fixed-hop expansion method for each query. However it helps us avoid bills for tokens that can happen with fixed-hop when a question requires many steps. We have set a limit of 15 steps and a timeout of 30 seconds per step. This gives us a worst-case scenario that we can share with users.

E. Educational utility

We did a study with five learners to test our tool. We asked them to rate it on a scale of 1 to 5. Here are the results: Concept Extraction scored 4.8 Flashcard Utility scored 4.7 Summarisation Quality scored 4.6 Exam Relevance scored 4.5 and Graph Coherence scored 4.4. Our sample size is small. The results are consistent, across all five areas.

VII. DISCUSSION

A. What we learned building this

When we started this project we did not think that how we used confidence would make such a difference. Just using C1, which gets rid of triples helps a little.. It does not make the community coherence much better. When we used C1 and C2 together it made a difference. This shows that confidence is really useful when it changes the topology, not the information we put in. We also found out that C3 makes a difference when the questions are harder. For questions it is okay to use fixed-hop.. For questions that have sub-questions, with different levels of difficulty the version guided by the Large Language Model works much better. C4 was not as noticeable. Its convergence rule turned out to be a good way to rank evidence almost by chance. The Large Language Model and confidence and topology and community coherence all matter when we use C1 and C2 and C3 and C4.

B. What we are not claiming

Some caveats:

- Our largest checked graph has 329 parts. Anything we say about scaling across the data set is just a guess, not a fact.
- The 0.40/0.35/0.25 weights in $\text{Conf}(t)$ are based on theory. Never tested against a correct labelled set. That is the obvious test.
- Everything depends on the LLM used. Results, from llama3.2:3b may not work well with models and we have not studied how well they work with different prompts.
- The confidence rules were made for content. We do not know if they work well for legal or general corpora.

C. What comes next

Four directions we want to explore are:

* Replacing the per-step Large Language Model call inside C3 with a learned policy. This should make things faster without losing flexibility.

* Linking entities across documents. This way we can build one graph that covers all our data instead of one graph per upload.

* We also want to do a study, with users to see if answers based on graphs really help students learn not just give them answers.

* Lastly we need to adjust the confidence weights based on labels that humans have annotated for quality.

VIII. CONCLUSION

The eduRAG system is built around an idea: the confidence that we have in the information we extract is important and it should be used throughout the entire GraphRAG pipeline. It should not be ignored after the step.

The eduRAG system really. We can see this with CW-Leiden. CW-Leiden is able to group things in a way that makes sense with a mean weighted modularity of 0.9248 and a mean community coherence of 0.3173 across three real document artifacts.

When we use the eduRAG system on the Educational QA corpus we get good results. The results are a ROUGE-L of 0.442 which's 25.6 percent better than before. We also get a BERTScore F1 of 0.912 which's 8.3 percent better and a METEOR of 0.385 which is 13.9 percent better than the baseline.

The entire eduRAG system is easy to reproduce from a YAML file. It runs like a real application, not just a notebook. We hope that the eduRAG system will be a starting point, for anyone who wants to use confidence-aware retrieval in a real way.

ACKNOWLEDGEMENTS

We want to thank the Department of Computer Engineering at AISSMS Institute of Information Technology in Pune. They gave us the computers we needed to do our work. They are still helping us. We also want to

thank the people who made NetworkX, leidenalg, ChromaDB sentence-transformers and FastAPI. These open-source communities and the tools they made were really important for our project. We could not have done it without the Department of Computer Engineering and the people, behind NetworkX, leidenalg, ChromaDB sentence-transformers and FastAPI.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [2] V. Karpukhin, B. Oğuz, S. Min *et al.*, "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020.
- [3] X. Ho, A.-K. Duong Nguyen, S. Sugawara, and A. Aizawa, "Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps," in *Proc. COLING*, 2020.
- [4] N. Reimers and I. Gurevych, "Making monolingual sentence embeddings multilingual using knowledge distillation," in *Proc. EMNLP*, 2020.
- [5] T. Brown, B. Mann, N. Ryder *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [6] P. Cabot and R. Navigli, "REBEL: Relation extraction by end-to-end language generation," in *Findings of EMNLP*, 2021.
- [7] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open domain question answering," in *Proc. EACL*, 2021.
- [8] H. Trivedi, M. Bauer, T. Khot, and A. Sabharwal, "MuSiQue: Multihop questions via single-hop question composition," *Trans. ACL*, vol. 10, pp. 539–554, 2022.
- [9] J. Wei, X. Wang, D. Schuurmans *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [10] O. Press, M. Zhang, S. Min *et al.*, "Measuring and narrowing the compositionality gap in language models," in *Findings of EMNLP*, 2023.
- [11] A. Asai, Z. Wu, Y. Wang *et al.*, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. ICLR*, 2024.
- [12] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. Park, "Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity," in *Proc. NAACL*, 2024.
- [13] D. Edge, H. Trinh, N. Cheng *et al.*, "From local to global: A graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.
- [14] B. Jin, C. Xie, J. Zhang *et al.*, "Graph chain-of-thought: Augmenting large language models by reasoning on graphs," in *Findings of ACL*, 2024.
- [15] Y. Gao, Y. Xiong, X. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2024.
- [16] L. Tan, B. Wang, B. Yu *et al.*, "Adaptive graph retrieval-augmented generation," in *Proc. NAACL*, 2024.
- [17] B. Sarmah, B. Hall, R. Rao *et al.*, "HybridRAG: Integrating knowledge graphs and vector retrieval augmented generation for efficient information extraction," in *Proc. ACM ICAIF*, 2024.
- [18] D. Sanmartin, "KG-RAG: Bridging the gap between knowledge and creativity," *arXiv preprint arXiv:2405.12035*, 2024.
- [19] B. Peng, Y. Zhu, Y. Liu *et al.*, "Graph retrieval-augmented generation: A survey," *ACM Trans. Information Systems*, 2025.
- [20] H. Han, Y. Wang, H. Shomer *et al.*, "Retrieval-augmented generation with graphs (GraphRAG)," *arXiv preprint arXiv:2501.00309*, 2025.
- [21] Y. Hu, Z. Lei, Z. Zhang *et al.*, "GRAG: Graph retrieval-augmented generation," in *Findings of NAACL*, 2025.
- [22] S. Knollmeyer, O. Caymazer, and D. Grossmann, "Document GraphRAG: Knowledge graph enhanced retrieval augmented generation for document question answering within the manufacturing domain," *Electronics*, vol. 14, no. 11, 2025.
- [23] A. Brown, M. Roman, and B. Devereux, "A systematic literature review of retrieval-augmented generation: Techniques, metrics, and challenges," *Big Data and Cognitive Computing*, vol. 9, no. 12, 2025.

- [24] Z. Guo, L. Xia, Y. Yu *et al.*, “LightRAG: Simple and fast retrieval-augmented generation,” in *Proc. ICLR*, 2025.
- [25] P. Zhao, H. Zhang, Q. Yu *et al.*, “Retrieval-augmented generation for AI-generated content: A survey,” *Data Science and Engineering*, vol. 11, pp. 1–29, 2026.